

Retirement Strategies System - Technical Documentation

This document details the complete decision tree, data sources, feasibility checks, and projection mathematics for the retirement strategies system.

1. Strategy Decision Tree

The retirement strategies system uses a **hierarchical, priority-ordered decision tree** with 4 strategy levels. Each strategy is evaluated in order with cumulative context tracking.

Strategy Priority Hierarchy

```

Priority 1: EMPLOYER MATCH (Maximise Free Money)
├── Check if employee contribution < employer matching limit
├── Infer matching limit from contribution patterns or explicit field
├── AFFORDABILITY CHECK:
│   ├── Calculate net cost of additional contribution
│   │   ├── First £2,000/year: Salary sacrifice (no cost -
│   │   │   saves tax AND NI)
│   │   └── Above £2,000/year: Relief at source (cost =
│   │       contribution × (1 - tax_rate))
│   ├── Compare net cost to disposable income
│   └── If NOT affordable:
│       ├── Show green message explaining situation
│       ├── Mark strategy as skipped_reason: 'affordability'
│       └── Skip directly to Priority 3 (bypass Priority 2)
└── If affordable: Recommend increasing employee contribution to
    capture full match

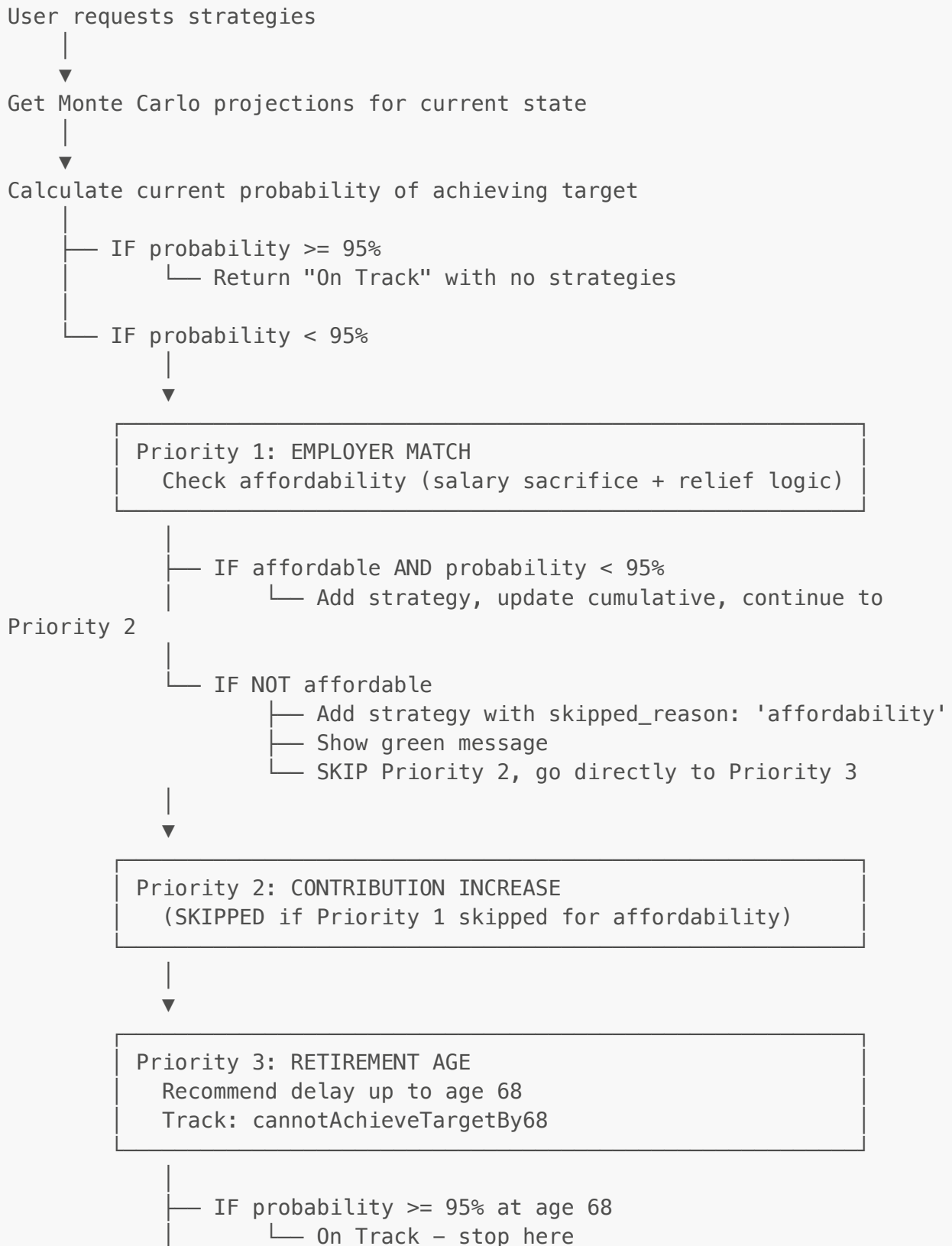
Priority 2: CONTRIBUTION INCREASE (Maximise Tax-Advantaged Saving)
├── SKIP if Priority 1 was skipped due to affordability (no message)
├── Check affordability (disposable income > 0)
├── Check Annual Allowance remaining
├── Include carry-forward if available
└── Recommend 50% of available capacity (balanced approach)

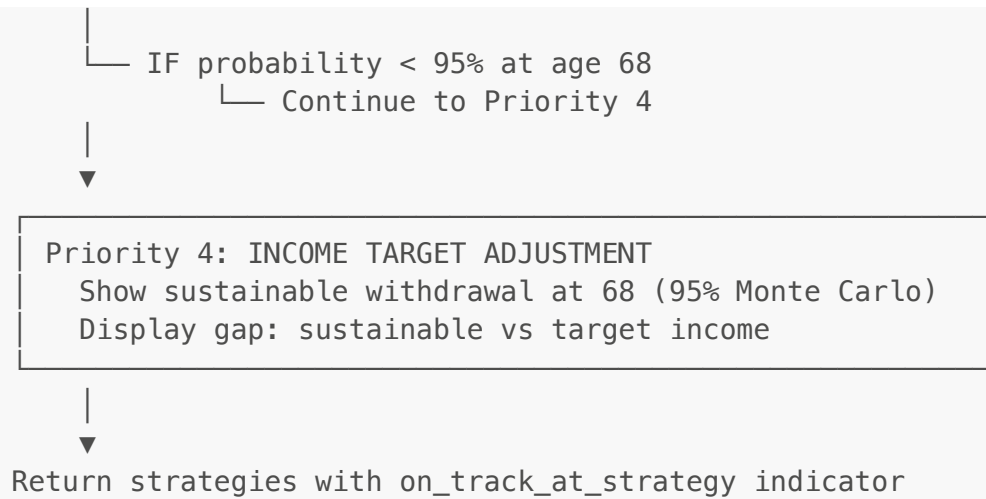
Priority 3: RETIREMENT AGE ADJUSTMENT (Extend Accumulation Period)
├── Find minimum delay that achieves 95% income coverage
├── Test years 1-3 incrementally
├── Cap recommendations at age 68, allow slider to 75
├── TRACK: If age 68 does NOT achieve target → trigger Priority 4
└── Provide pot projection at delayed retirement

Priority 4: INCOME TARGET REDUCTION (Adjust Expectations)
├── Triggered if:
│   ├── Other strategies insufficient, OR
│   └── Priority 3 cannot achieve target by age 68
  
```

- └─ Calculate sustainable withdrawal at age 68 (95% Monte Carlo)
- └─ Show GAP: difference between sustainable income and target
 - └─ "You can sustainably withdraw £X/year. This is £Y/year less than your target of £Z/year."
- └─ Use green-toned messaging (informative, not alarming)

Decision Flow





Key Decision Logic

File: `app/Services/Retirement/RetirementStrategyService.php` (lines 36-206)

```

public function getStrategies(int $userId): array
{
    // 1. Get current projection status
    $projections = $this->projectionService->getProjections($userId);
    $currentProbability = $projections['income_drawdown']['probability'];

    // 2. If on track, no strategies needed
    if ($currentProbability >= 95) {
        return ['on_track' => true, 'strategies' => []];
    }

    // 3. Build strategies with cumulative context
    $strategies = [];
    $cumulativeMonthly = 0;
    $cumulativeIncome = 0;

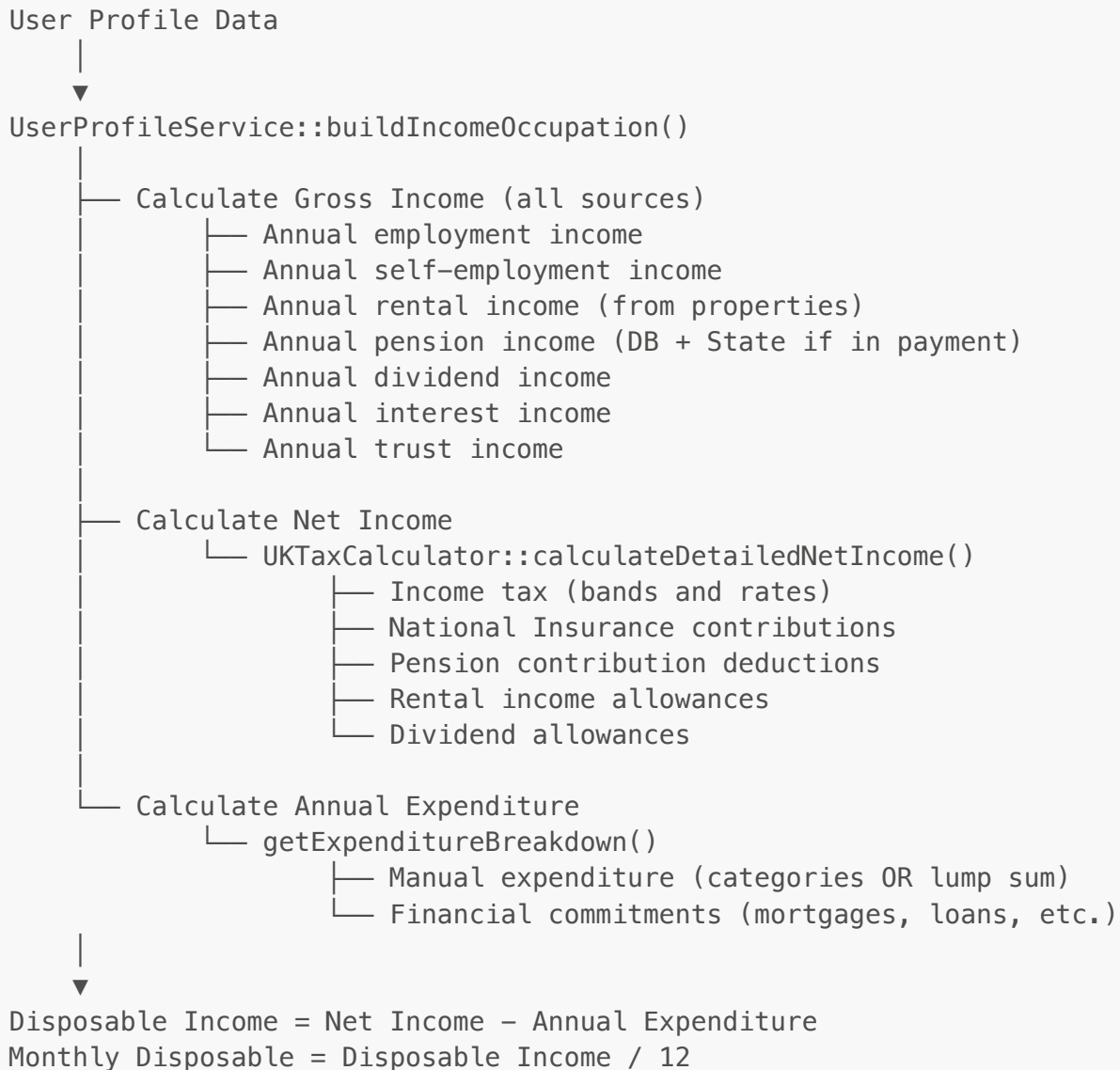
    // Priority 1: Employer Match
    $employerMatch = $this->checkEmployerMatchStrategy($userId, ...);
    if ($employerMatch) {
        $strategies[] = $employerMatch;
        $cumulativeMonthly += $employerMatch['additional_monthly'];
        $cumulativeIncome += $employerMatch['additional_annual_income'];
    }

    // Priority 2: Contribution Increase (with cumulative context)
    // Priority 3: Retirement Age Adjustment
    // Priority 4: Income Target Reduction

    return $strategies;
}
  
```

2. Disposable Income Source

Complete Data Flow



Expenditure Calculation

File: `app/Services/UserProfile/UserProfileService.php` (lines 177-228)

```

// Two modes of expenditure entry:

IF expenditure_entry_mode === 'category':
    monthly_manual = SUM(
        food_groceries,
        transport_fuel,
        healthcare_medical,
        insurance,
        mobile_phones,
        internet_tv,
        subscriptions,

```

```
        clothing_personal_care,
        entertainment_dining,
        holidays_travel,
        pets,
        childcare,
        school_fees,
        school_lunches,
        school_extras,
        university_fees,
        children_activities,
        gifts_charity,
        regular_savings,
        other_expenditure
    )
ELSE:
    monthly_manual = monthly_expenditure (simple lump sum mode)

// Add financial commitments
monthly_commitments = getFinancialCommitments(user)
    |— Mortgage payments
    |— Loan repayments
    |— Insurance premiums
    |— Other committed outgoings

monthly_total = monthly_manual + monthly_commitments
annual_total = monthly_total * 12

// Final calculation
disposable_income = net_income - annual_total
monthly_disposable = disposable_income / 12
```

Tax Calculation Integration

The net income calculation uses `UKTaxCalculator::calculateDetailedNetIncome()` which factors in:

Component	Treatment
Income Tax	Applied by band (20%, 40%, 45%)
National Insurance	Employee NI rates
Pension Contributions	Reduce taxable income (relief at marginal rate)
Rental Allowances	Property allowance, finance cost restrictions
Dividend Allowances	£1,000 dividend allowance (2024/25)

3. Feasibility Checks

Affordability Validation

File: `app/Services/Retirement/RetirementStrategyService.php` (lines 546-631)

For contribution increase strategies:

```
// 1. Must have positive disposable income
if ($disposable_income <= 0) {
    return null; // Strategy not viable
}

// 2. Must have remaining Annual Allowance
if ($remaining_allowance <= 0) {
    return null; // Strategy not viable
}

// 3. Calculate maximum additional contribution
$max_additional_annual = min($disposable_income, $remaining_allowance);
$max_additional_monthly = $max_additional_annual / 12;

// 4. Recommend half of maximum (conservative approach)
$recommended_monthly = $max_additional_monthly * 0.5;

// 5. Identify binding constraint for user messaging
if ($remaining_allowance < $disposable_income) {
    $constraint = "Limited by Annual Allowance";
} else {
    $constraint = "Limited by affordability";
}
```

Annual Allowance Checks

File: `app/Services/Retirement/AnnualAllowanceChecker.php`

```
// Standard values (from TaxConfigService)
$standardAA = 60000; // £60,000
$minimumTaperedAA = 10000; // £10,000
$thresholdIncome = 260000; // £260,000
$adjustedIncomeThreshold = 312000; // £312,000

// Tapering calculation for high earners
if ($income > $thresholdIncome && $adjustedIncome >
$adjustedIncomeThreshold) {
    $isTapered = true;
    $reduction = ($adjustedIncome - $adjustedIncomeThreshold) / 2;
    $availableAllowance = max($minimumTaperedAA, $standardAA -
$reduction);
} else {
    $availableAllowance = $standardAA;
}

// Calculate remaining
```

```

$currentContributions = sum(employee + employer contributions);
$remaining = $availableAllowance - $currentContributions;

// Carry-forward (estimated - full tracking requires historical data)
$carryForward = 60000; // 1 year estimate

```

Employer Match Affordability Check

Tax Year 2025/26 Rules:

The net cost of additional pension contributions depends on how the contribution is made:

Method	Limit	Income Tax	National Insurance	Net Cost
Salary Sacrifice	£2,000/year	Saved (full relief)	Saved (full relief)	£0
Relief at Source	Above £2,000	Relief at marginal rate	Still payable	contribution × (1 - tax_rate)

Net Cost Calculation:

```

function calculateNetCostOfContribution($additionalAnnual, $user) {
    $salarySacrificeLimit = 2000.0;

    // First £2,000 via salary sacrifice - no cost to employee
    $viaSalarySacrifice = min($additionalAnnual, $salarySacrificeLimit);
    $viaReliefAtSource = max(0, $additionalAnnual -
    $salarySacrificeLimit);

    // Get marginal tax rate based on gross income
    $marginalRate = getMarginalTaxRate($user);

    // Relief at source: employee pays net (after tax relief)
    // Basic rate (20%): cost = contribution × 0.80
    // Higher rate (40%): cost = contribution × 0.60
    // Additional rate (45%): cost = contribution × 0.55
    $netCostReliefAtSource = $viaReliefAtSource * (1 - $marginalRate);

    return $netCostReliefAtSource; // Salary sacrifice portion has zero
cost
}

function getMarginalTaxRate($user) {
    $grossIncome = $user->annual_employment_income + $user->
annual_self_employment_income;
    $personalAllowance = 12570;
    $basicLimit = 50270;
    $higherLimit = 125140;

```

```

    if ($grossIncome <= $personalAllowance) return 0.0;
    if ($grossIncome <= $basicLimit) return 0.20;
    if ($grossIncome <= $higherLimit) return 0.40;
    return 0.45;
}

```

Affordability Decision:

```

$additionalAnnual = $additionalMonthly * 12;
$netCost = calculateNetCostOfContribution($additionalAnnual, $user);
$canAfford = $affordability['disposable_income'] >= $netCost;

if (!$canAfford) {
    // Return strategy with:
    // - applicable: true (strategy exists but skipped)
    // - skipped_reason: 'affordability'
    // - affordability_message: Green-toned message explaining situation
    // - Skip directly to Priority 3
}

```

Green Message When Not Affordable:

"While maximising your employer's matching contribution would significantly boost your pension, it's not currently affordable based on your disposable income. Consider reviewing your monthly expenditure or revisiting this strategy when your financial circumstances change."

Employer Match Validation

File: `app/Services/Retirement/RetirementStrategyService.php` (lines 514-542)

```

// Infer matching limit robustly
function inferEmployerMatchLimit($pension) {
    // Priority 1: Explicit limit set by user (1-50%)
    if ($pension->employer_matching_limit && $pension->employer_matching_limit > 0) {
        return $pension->employer_matching_limit;
    }

    // Priority 2: Infer from contribution pattern
    if ($pension->employer_contribution_percent > $pension->employee_contribution_percent) {
        // Employer contributes more - employee can increase to match
        return $pension->employer_contribution_percent;
    }
}

```



```

    }

    // Priority 3: Already at or above employer level
    if ($pension->employee_contribution_percent >= $pension->employer_contribution_percent) {
        return null; // No match opportunity
    }

    return null; // No valid matching scheme detected
}

// Minimum gap to show strategy (avoid trivial improvements)
$minimumGap = 0.5; // 0.5%
if ($matchingOpportunity < $minimumGap) {
    return null;
}

```

4. Strategy Projections - Mathematics

Base Formula

All strategy projections use the **Future Value of Annuity** formula:

$$FV = PMT \times [((1 + r)^n - 1) / r]$$

Where:

- **FV** = Future Value of additional contributions at retirement
- **PMT** = Monthly contribution amount
- **r** = Monthly growth rate (annual rate / 12)
- **n** = Number of months to retirement

Projection Calculation

File: `app/Services/Retirement/RetirementStrategyService.php` (lines 982-1441)

```

// Year-by-year pot growth with strategy
for ($year = 0; $year <= $yearsToRetirement; $year++) {
    if ($year == 0) {
        $potWithoutStrategy = $currentPot;
        $potWithStrategy = $currentPot;
    } else {
        // Without strategy: from Monte Carlo 5th percentile
        $potWithoutStrategy = $monteCarloData[$year - 1]['percentile_5'];

        // With strategy: add additional pot from new contributions
        $months = $year * 12;
        $monthlyRate = $growthRate / 12;
    }
}

```

```

    $additionalPot = $additionalMonthly *
      ((pow(1 + $monthlyRate, $months) - 1) / $monthlyRate);

    $potWithStrategy = $potWithoutStrategy + $additionalPot;
  }

  $projection[] = [
    'year' => $calendarYear,
    'years_from_now' => $year,
    'pot_with_strategy' => round($potWithStrategy),
    'pot_without_strategy' => round($potWithoutStrategy)
  ];
}

```

Retirement Income Calculation

```

// Sustainable withdrawal rate: 4.7%
$withdrawalRate = 0.047;

// Calculate sustainable income from DC pot
$sustainableIncomeWith = $potAtRetirementWith * $withdrawalRate;
$sustainableIncomeWithout = $potAtRetirementWithout * $withdrawalRate;

// Add guaranteed income (DB + State Pension)
$guaranteedIncome = $dbPensionAnnual + $statePensionAnnual;

$totalIncomeWith = $sustainableIncomeWith + $guaranteedIncome;
$totalIncomeWithout = $sustainableIncomeWithout + $guaranteedIncome;

// Calculate coverage percentage
$coveragePercent = ($totalIncomeWith / $targetIncome) * 100;

```

Probability Calculation

```

// Income ratio drives probability
$incomeRatio = $projectedIncome / $targetIncome;

// Linear mapping with caps
$probability = 10 + ($incomeRatio * 85);
$probability = min(95, max(10, $probability));

// Probability bands:
// 100%+ of target → 95% (Excellent)
// 90–99% of target → 85% (On Track)
// 75–89% of target → 65% (Needs Attention)
// 50–74% of target → 40% (Off Track)
// 25–49% of target → 20% (Significantly Off Track)
// <25% of target → 10% (Critical)

```

```
// Longevity bonus
if ($fundLastsYears >= 35) {
  $probability += 5;
}
if ($fundLastsYears >= 25) {
  $probability += 3;
}
```

Employer Match Strategy Math

```
// Additional income from increasing employee contribution
// (which also triggers additional employer match)

$currentEmployeePercent = $pension->employee_contribution_percent;
$matchingLimit = $inferredMatchingLimit;
$salary = $pension->annual_salary;

// Calculate additional contribution
$additionalPercent = $matchingLimit - $currentEmployeePercent;
$additionalMonthly = ($salary * $additionalPercent / 100) / 12;

// Double it because employer matches
$totalAdditionalMonthly = $additionalMonthly * 2;

// Project to retirement
$months = $yearsToRetirement * 12;
$additionalPot = $totalAdditionalMonthly *
  ((pow(1 + $monthlyRate, $months) - 1) / $monthlyRate);

$additionalIncome = $additionalPot * 0.047;
```

5. Data Flow - Complete End-to-End

API Request Flow

FRONTEND: StrategiesTab.vue
User views Retirement → Strategies tab



API: GET /api/retirement/strategies



STEP 1: GET PROJECTIONS

```
RetirementProjectionService::getProjections($userId)
├─ Run Monte Carlo (1,000 iterations)
├─ Extract probability bands (5th-90th percentiles)
├─ Calculate year-by-year projections (0-30 years)
└─ Return: pension_pot_projection + income_drawdown
```

**STEP 2: CALCULATE AFFORDABILITY**

```
calculateAffordability($user)
├─ UserProfileService::getCompleteProfile()
├─ Get income_occupation with net income
├─ Get expenditure_breakdown (categories + commitments)
├─ Calculate: disposable = net - expenditure
└─ Return: gross, net, expenditure, disposable
```

**STEP 3: CHECK ANNUAL ALLOWANCE**

```
AnnualAllowanceChecker::checkAnnualAllowance()
├─ Sum all DC pension contributions
├─ Get user income for tapering check
├─ Apply tapering if high earner
├─ Estimate carry-forward
└─ Return: standard, available, remaining, carry_forward
```

**STEP 4: BUILD STRATEGIES (WITH CUMULATIVE CONTEXT)**

```
FOR each strategy priority (1-4):
├─ Calculate additional monthly contribution
├─ Calculate additional annual income
├─ Calculate new probability
├─ Build projection with cumulative values
├─ Add to strategies list
├─ Update cumulative values for next strategy
└─ Break if probability >= 95%
```

**STEP 5: RETURN STRUCTURED RESPONSE**

```
{
  current_status: { probability, income_gap, ... },
  affordability: { disposable_income, ... },
  annual_allowance: { remaining, ... },
  strategies: [ {impact, projection}, ... ],
  on_track_at_strategy: 2
}
```



FRONTEND RENDERING

- Summary cards (disposable income, AA remaining)
- Strategy cards with interactive sliders
- ApexCharts: pot growth comparison
- Income comparison tables

API Response Structure

```
{
  "current_status": {
    "on_track_status": "Off Track",
    "probability": 72,
    "projected_income": 24500,
    "target_income": 40000,
    "income_gap": 15500
  },
  "affordability": {
    "gross_income": 80000,
    "net_income": 62000,
    "annual_expenditure": 45000,
    "monthly_disposable": 1416.67,
    "disposable_income": 17000
  },
  "annual_allowance": {
    "standard_allowance": 60000,
    "available_allowance": 60000,
    "remaining_allowance": 55000,
    "current_contributions": 5000
  },
  "strategies": [
    {
      "type": "employer_match",
      "priority": 1,
      "title": "Maximise Employer Match",
      "current_value": 5,
      "recommended_value": 8,
      "slider_config": {
        "min": 5,
        "max": 10,
        "step": 0.5
      },
      "impact": {
        "additional_monthly": 180,
        "additional_annual_income": 1125,
        "new_probability": 78,
        "probability_improvement": 6
      }
    }
  ]
}
```

```
        "projection": {
            "pot_growth": [...],
            "with_strategy": {...},
            "without_strategy": {...}
        }
    },
    "on_track_at_strategy": 3
}
```

6. Key Files Reference

File	Purpose
app/Services/Retirement/RetirementStrategyService.php	Main strategy logic, decision tree, projections
app/Services/Retirement/RetirementProjectionService.php	Monte Carlo projections, income drawdown
app/Services/Retirement/AnnualAllowanceChecker.php	AA calculations, tapering, carry-forward
app/Services/UserProfile/UserProfileService.php	Disposable income, expenditure breakdown
app/Services/Tax/UKTaxCalculator.php	Net income calculation
app/Http/Controllers/Api/RetirementController.php	API endpoints
resources/js/components/Retirement/StrategiesTab.vue	Frontend strategy display
resources/js/components/Retirement/StrategyCard.vue	Individual strategy card with slider

7. Mathematical Formulas Summary

1. FUTURE VALUE OF ANNUITY (monthly contributions):
FV = PMT × (((1 + r)^n - 1) / r)
Where:
PMT = monthly contribution
r = monthly_return = annual_return / 12
n = months to retirement
2. RETIREMENT INCOME FROM POT:
annual_income = pot_value × 0.047 (4.7% withdrawal rate)
3. PROBABILITY CALCULATION:
income_ratio = projected_income / target_income
probability = 10 + (income_ratio × 85)

```
CAPPED: MIN(95, MAX(10, probability))
```

4. EMPLOYER MATCH IMPACT:

```
additional_income = (salary × additional_percent / 100) × 2  
(doubled because employer matches employee increase)
```

5. DISPOSABLE INCOME:

```
disposable = net_income - (manual_expenditure + financial_commitments)
```

6. ANNUAL ALLOWANCE TAPERING:

```
reduction = (adjusted_income - £312,000) / 2  
tapered_aa = MAX(£10,000, £60,000 - reduction)
```

IMPLEMENTED: Priority 2 Enhancement - Contribution Increase with Tax Relief Reinvestment

Status: Implemented **Date:** 22 January 2026

Overview

Enhance Priority 2 (Contribution Increase) strategy to:

1. Calculate optimal contribution amount considering relief at source
2. Account for self-assessment refund for higher/additional rate taxpayers
3. Plan reinvestment of tax refund into pension or investment strategy
4. Project compound benefit of reinvestment until retirement age

Current Priority 2 Logic (Summary)

```
IF employer match not skipped for affordability:  
  |— Check disposable income > 0  
  |— Check Annual Allowance remaining > 0  
  |— max_additional = min(disposable_income, remaining_allowance)  
  |— Recommend 50% of max_additional
```

Problem: This doesn't account for:

- The 20% HMRC uplift (relief at source) that increases buying power
- Higher/additional rate self-assessment refunds that can be reinvested
- Optimal timing of reinvestment (pension vs ISA/Bond/GIA)
- Compound benefit of systematic reinvestment until retirement

Proposed Priority 2 Logic

Step 1: Entry Conditions

```

IF employer match already maxed OR employer match applied successfully:
  └─ Proceed to Priority 2

IF employer match skipped due to affordability:
  └─ SKIP Priority 2 entirely (go to Priority 3)

```

Step 2: Affordability Check

```

Check user's remaining disposable income (after any employer match
contribution increase)
IF disposable_income <= 0:
  └─ SKIP Priority 2 (no message)

```

Step 3: Annual Allowance Check

```

Get remaining Annual Allowance:
  total_allowance = £60,000 (or tapered amount for high earners)
  current_contributions = employee + employer contributions (all
pensions)
  remaining_allowance = total_allowance - current_contributions +
carry_forward

IF remaining_allowance <= 0:
  └─ SKIP pension contribution, suggest investment priority order (see
Step 6)

```

Step 4: Calculate Optimal Contribution

Relief at Source Mechanics:

What User Pays	HMRC Adds (20%)	Gross in Pension	User's Tax Band	Self-Assessment Refund
£80	£20	£100	Basic (20%)	£0
£80	£20	£100	Higher (40%)	£20 (additional 20%)
£80	£20	£100	Additional (45%)	£25 (additional 25%)

Effective Cost Formula:

```

For a gross contribution of £G:

Basic rate taxpayer (20%):

```



```

net_cost = G × 0.80
refund = £0
effective_cost = G × 0.80

```

Higher rate taxpayer (40%):

```

net_cost = G × 0.80 (paid upfront)
refund = G × 0.20 (via self-assessment)
effective_cost = G × 0.60

```

Additional rate taxpayer (45%):

```

net_cost = G × 0.80 (paid upfront)
refund = G × 0.25 (via self-assessment)
effective_cost = G × 0.55

```

Maximum Contribution Calculation:

```

// User can afford to pay this much upfront (net)
$maxNetPayment = $disposable_income;

// This translates to a gross pension contribution of:
$maxGrossContribution = $maxNetPayment / 0.80; // Relief at source adds
25%

// But cap at remaining Annual Allowance
$recommendedGross = min($maxGrossContribution, $remaining_allowance);

// Calculate what user actually pays upfront
$userPaysUpfront = $recommendedGross * 0.80;

// Calculate self-assessment refund for higher/additional rate
if ($marginalRate == 0.40) {
    $selfAssessmentRefund = $recommendedGross * 0.20;
} elseif ($marginalRate == 0.45) {
    $selfAssessmentRefund = $recommendedGross * 0.25;
} else {
    $selfAssessmentRefund = 0;
}

```

Step 5: Self-Assessment Refund Reinvestment Strategy

The self-assessment refund arrives approximately January following the tax year end (e.g., for 2025/26 tax year, refund arrives ~January 2027).

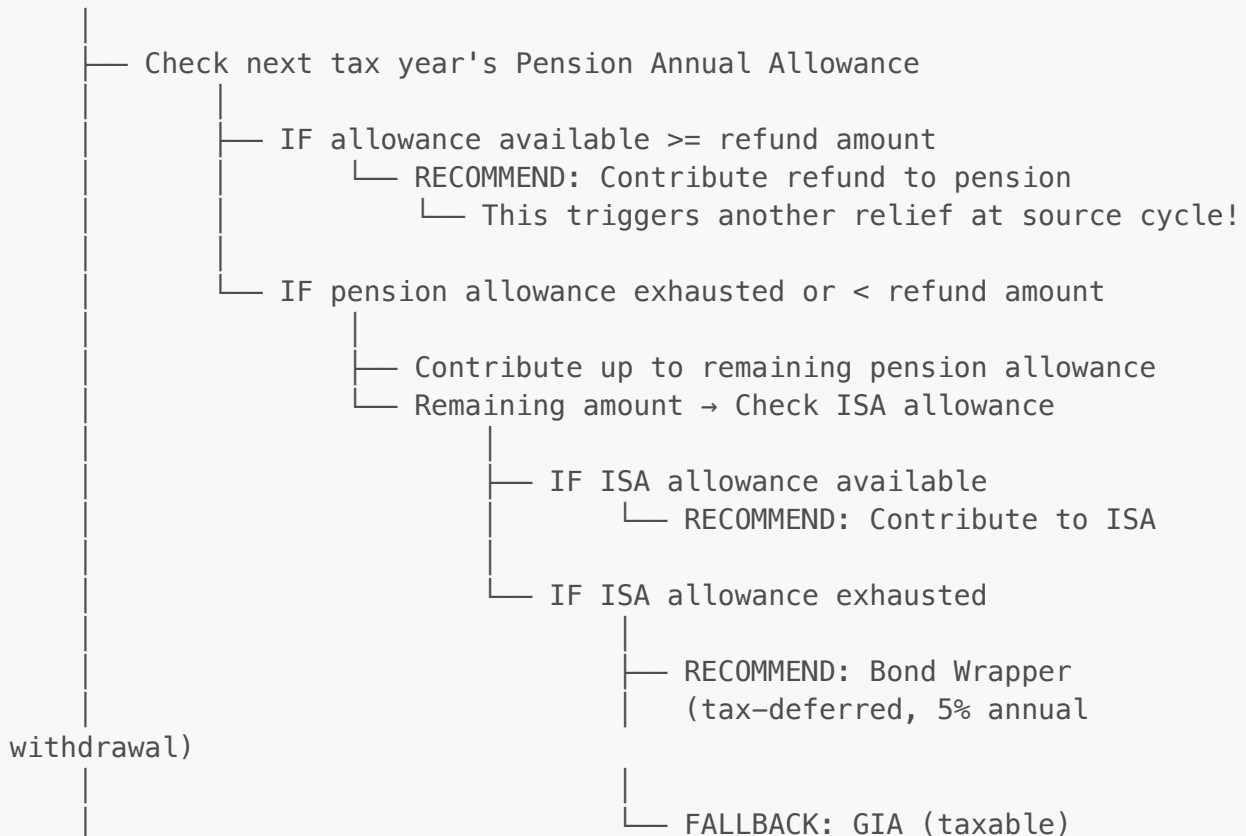
Investment Priority Order (when pension allowance exhausted):

1. PENSION – If Annual Allowance available (tax relief + tax-free growth)
2. ISA – If £20k allowance available (tax-free growth, flexible access)

- 3. BOND WRAPPER – Onshore/Offshore bond (tax-deferred growth, 5% withdrawal rule)
- 4. GIA – General Investment Account (no limit, but fully taxable)

Decision Tree for Refund:

When self-assessment refund is received:



Step 6: Compound Projection Until Retirement

Project the compound benefit of systematic refund reinvestment from current age until retirement age.

Projection Formula:

```

$yearsToRetirement = $user->target_retirement_age - $user->current_age;
$annualGrossContribution = $recommendedGross;
$marginalRate = getMarginalTaxRate($user);
$growthRate = 0.05; // 5% assumed growth

$projectedValues = [];
$cumulativeRefundReinvested = 0;

for ($year = 1; $year <= $yearsToRetirement; $year++) {
    // Year 1: Initial contribution
    // Year 2+: Initial contribution + reinvested refund from previous year
  
```

```
$refundFromPreviousYear = $cumulativeRefundReinvested;

// Reinvest refund into pension (with its own relief at source)
$reinvestedGross = $refundFromPreviousYear / 0.80; // Net becomes
gross

// Total gross contribution this year
$totalGrossThisYear = $annualGrossContribution + $reinvestedGross;

// Calculate this year's refund (for next year's reinvestment)
if ($marginalRate == 0.40) {
    $thisYearRefund = $totalGrossThisYear * 0.20;
} elseif ($marginalRate == 0.45) {
    $thisYearRefund = $totalGrossThisYear * 0.25;
} else {
    $thisYearRefund = 0;
}

$cumulativeRefundReinvested = $thisYearRefund;

// Project pot value with growth
$projectedValues[$year] = [
    'gross_contribution' => $totalGrossThisYear,
    'refund_reinvested' => $refundFromPreviousYear,
    'refund_generated' => $thisYearRefund,
    'pot_value' => calculateFutureValue($totalGrossThisYear,
    $growthRate, $yearsToRetirement - $year)
];

// Total additional pot at retirement from this strategy
$totalAdditionalPot = array_sum(array_column($projectedValues,
'pot_value'));
```

Example: Higher Rate Taxpayer, 20 Years to Retirement

Year	Base Contribution	Refund Reinvested	Total Gross	Refund Generated
1	£10,000	£0	£10,000	£2,000
2	£10,000	£2,000 → £2,500 gross	£12,500	£2,500
3	£10,000	£2,500 → £3,125 gross	£13,125	£2,625
...	...	...	...	...
20	£10,000	£X → £Y gross	£Z	£W

Compound Benefit Summary:

- Without reinvestment: £200,000 total contributions over 20 years
- With reinvestment: £280,000+ total contributions (40%+ more)
- Additional pot at retirement: £X (shown to user)

API Response Structure

```
{
  "type": "increase_contribution",
  "priority": 2,
  "title": "Increase Pension Contributions",
  "description": "Increase your pension contributions to boost your retirement pot.",
  "contribution_breakdown": {
    "gross_contribution": 10000,
    "user_pays_upfront": 8000,
    "hmrc_adds": 2000,
    "self_assessment_refund": 2000,
    "effective_annual_cost": 6000,
    "tax_band": "higher"
  },
  "refund_reinvestment": {
    "refund_amount": 2000,
    "refund_timing": "January 2027",
    "recommended_destination": "pension",
    "pension_allowance_available": true,
    "fallback_order": ["pension", "isa", "bond_wrapper", "gia"]
  },
  "compound_projection": {
    "years_to_retirement": 20,
    "without_reinvestment": {
      "total_contributions": 200000,
      "projected_pot": 350000
    },
    "with_reinvestment": {
      "total_contributions": 280000,
      "projected_pot": 490000,
      "additional_benefit": 140000
    },
    "yearly_breakdown": [
      { "year": 1, "gross": 10000, "refund_reinvested": 0, "refund_generated": 2000 },
      { "year": 2, "gross": 12500, "refund_reinvested": 2500, "refund_generated": 2500 },
      "...",
    ]
  },
  "constraints": {
    "limited_by": "affordability",
    "remaining_allowance": 45000,
    "disposable_income": 12000
  },
  "slider_config": {
    "min": 0,
    "max": 15000,
    "step": 500,
  }
}
```

```
    "unit": "/year gross",  
    "format": "currency"  
  }  
}
```

User Messaging

For Basic Rate Taxpayers:

"You can contribute up to £10,000/year to your pension. You pay £8,000 and HMRC automatically adds £2,000 (20% basic rate relief), giving you £10,000 in your pension."

For Higher Rate Taxpayers:

"You can contribute up to £10,000/year to your pension. You pay £8,000 upfront, HMRC adds £2,000 (20% relief at source), and you'll receive an additional £2,000 back through self-assessment (higher rate relief). Your effective cost is just £6,000 for £10,000 of pension contributions."

By reinvesting your tax refund each year until retirement (20 years), you could contribute an additional £80,000 to your pension at no extra cost, resulting in an estimated additional £140,000 in your retirement pot."

For Additional Rate Taxpayers:

"You can contribute up to £10,000/year to your pension. You pay £8,000 upfront, HMRC adds £2,000 (20% relief at source), and you'll receive an additional £2,500 back through self-assessment (additional rate relief). Your effective cost is just £5,500 for £10,000 of pension contributions."

By reinvesting your tax refund each year until retirement, you maximise the compound benefit of tax relief."

When Pension Allowance Exhausted:

```
"Your pension Annual Allowance is fully utilised. We recommend investing
your
tax refund in this order:
1. ISA (£20,000/year – tax-free growth)
2. Bond Wrapper (tax-deferred growth with 5% annual withdrawal allowance)
3. GIA (unlimited, but investment gains are taxable)"
```

Files to Modify

1. `app/Services/Retirement/RetirementStrategyService.php`
 - Update `checkContributionIncreaseStrategy()` method
 - Add `calculateContributionWithRelief()` method
 - Add `calculateRefundReinvestmentStrategy()` method
 - Add `projectCompoundBenefitToRetirement()` method
2. `resources/js/components/Retirement/StrategyCard.vue` (potentially)
 - Display contribution breakdown
 - Show compound projection chart
 - Show refund reinvestment recommendation

Implementation Checklist

- ☐ Update `checkContributionIncreaseStrategy()` to calculate relief at source
- ☐ Calculate gross contribution based on net affordability ($\div 0.80$)
- ☐ Calculate self-assessment refund for higher/additional rate taxpayers
- ☐ Determine refund reinvestment destination (pension → ISA → bond wrapper → GIA)
- ☐ Project compound benefit from current year to retirement age
- ☐ Update API response with `contribution_breakdown`
- ☐ Update API response with `refund_reinvestment` recommendation
- ☐ Update API response with `compound_projection` to retirement
- ☐ Update user messaging for each tax band
- ☐ Add tests for each tax band scenario
- ☐ Add tests for compound projection accuracy
- ☐ Update `pensionStrat.md` with final implementation details

Design Decisions

1. **Slider behaviour:** Shows gross contribution (what goes into pension) with net cost displayed separately
2. **Refund timing:** Model actual cash flow - "Refund arrives January 2027" for transparency
3. **Compound projection:** Project full reinvestment cycle until user's target retirement age

4. **Investment fallback order:** Pension → ISA → Bond Wrapper → GIA (in order of tax efficiency)

Verification Plan

1. **Basic Rate Taxpayer (James Carter):**

- Annual income ~£62,000 (basic rate)
- Verify: User pays = gross × 0.80
- Verify: No self-assessment refund shown
- Verify: Compound projection shows contributions only (no refund cycle)

2. **Higher Rate Taxpayer (David Mitchell):**

- Annual income ~£150,000 (higher rate)
- Verify: Self-assessment refund = gross × 0.20
- Verify: Refund reinvestment recommendation shown
- Verify: Compound projection shows increasing contributions each year

3. **Compound Projection Accuracy:**

- Test 10-year, 20-year, 30-year projections
- Verify growth calculations match expected FV formula
- Verify total additional pot at retirement is calculated correctly

4. **Allowance Exhausted Scenario:**

- Create test user with high contributions
- Verify: ISA recommended first
- Verify: Bond wrapper recommended when ISA exhausted
- Verify: GIA as final fallback